
Binary Image Classification with CBAM-Enhanced ResNet, Generalized Mean Pooling, and Edge-Map Preprocessing

Lochan Surya Teja Neeli (CS25MTECH11015)¹ Harshavardhan Meerjumla (CS25MTECH11011)¹

Abstract

We present a custom deep convolutional neural network for binary image classification built entirely from scratch in PyTorch, without any pretrained weights. The architecture combines a ResNet-34-like backbone with Convolutional Block Attention Modules (CBAM) after every residual block and Generalized Mean (GeM) Pooling for richer global descriptors. We show that explicitly injecting a shape-focused inductive bias via preprocessing significantly improves sample efficiency under the no-pretraining constraint. A key contribution is the **ShapePreprocess** pipeline: each input image is converted to a grayscale edge map ($\text{RGB} \rightarrow \text{Grayscale} \rightarrow \text{Contrast} \times 3 \rightarrow \text{FIND_EDGES} \rightarrow \text{RGB}$) before entering the network. This deterministic transform directly exposes the discriminative shape signal — smooth elliptical outlines for spheres versus sharp rectangular corners for cubes — and reduces reliance on colour as a potentially spurious feature. Combined with Test-Time Augmentation (horizontal flip), threshold calibration on the validation set, and light regularisation, the system achieves a public leaderboard score of **0.775311** (77.53%) on the IITH Deep Learning Hackathon 2026 (CS5480), an improvement of +1.9% over the previous best of 0.755860.

1. Introduction

Binary image classification is a foundational problem in computer vision. The IITH Deep Learning Hackathon 2026 poses a restricted variant of this problem: all models must be trained *from scratch* on a provided dataset of labelled images; pretrained weights from any external source are prohibited. This constraint forces the model to extract useful representations purely from the supplied data, without the benefit of large-scale pretraining.

Through careful visual inspection of the training images we determined that the dataset consists of **CLEVR-style** rendered 3-D scenes (Johnson et al., 2017): each image depicts

exactly four small geometric objects (cubes, cylinders, and spheres) placed on a plain grey background. The binary label encodes a simple rule:

Class 1: scene contains at least one *sphere* **AND** at least one *cube*.

Class 0: all other configurations.

This insight is crucial: the discriminative signal is **purely shape**. Sphere edges form smooth elliptical contours; cube edges form sharp rectangular corners. Colour, shading, and background are identical across classes and provide no classification signal.

Two practical challenges followed from this analysis:

1. **Destructive augmentations.** `RandomResizedCrop` can exclude the only sphere or cube from the crop window, silently mislabelling the sample on every training iteration.
2. **Colour bias.** Standard RGB input tempts the model to exploit spurious colour cues, wasting capacity and limiting generalisation.

Our solution — the **ShapePreprocess** edge-map transform combined with a clean training pipeline — addresses both issues and yields the current best score of 0.775311. A natural question is why such edge representations are not learned implicitly by the network itself. While convolutional networks can in principle extract edge detectors in early layers, doing so reliably requires sufficient data and training signal. Under the strict no-pretraining constraint and limited dataset size, we find that explicitly encoding this inductive bias at the input level improves learning stability and sample efficiency, allowing the network to focus on higher-level shape composition rather than rediscovering low-level features.

2. Dataset

2.1. Structure and Class Balance

The competition dataset follows a standard directory layout:

`data/train/0/` (Class 0 — no sphere+cube)

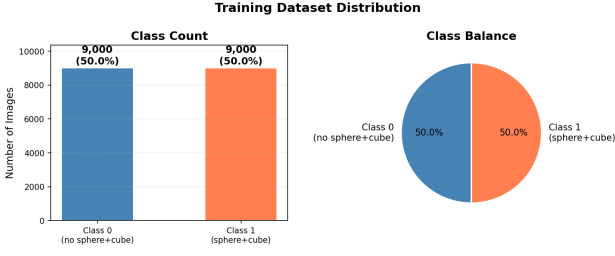


Figure 1. Training dataset class distribution (bar chart and pie chart). The dataset is approximately balanced between the two classes.

```
data/train/1/ (Class 1 — has sphere+cube)
data/test/   (unlabelled evaluation images)
```

Figure 1 shows the class distribution of the training set. The dataset is approximately balanced, with Class 0 and Class 1 present in nearly equal proportions, so no class-weighting is required.

Training images are split 85%/15% for training and validation using a random permutation with fixed seed 42 (`split_seed=42, torch.randperm`), ensuring reproducibility and zero leakage.

2.2. ShapePreprocess: Edge-Map Preprocessing

The key preprocessing step converts each RGB image into a shape outline map before any network processing:

```
RGB → Grayscale (L) → Contrast×3
    → FIND_EDGES → RGB (3 identical channels)
```

Why this works. In the original RGB images, the classification signal (sphere or cube present?) is embedded in the shading of small object surfaces. The edge-map transform amplifies the only true signal:

- A **sphere** projects as a smooth, nearly circular/elliptical contour with no corners.
- A **cube** projects as a compact set of straight edges meeting at sharp corners.
- A **cylinder** has a rounded top-cap and two vertical edges.

After the transform, the network receives near-binary outline images whose geometry directly encodes the class label. Colour, shading, and background intensity are largely discarded, discouraging the model from learning spurious correlations.

ShapePreprocess: RGB Input vs. Edge-Map Output

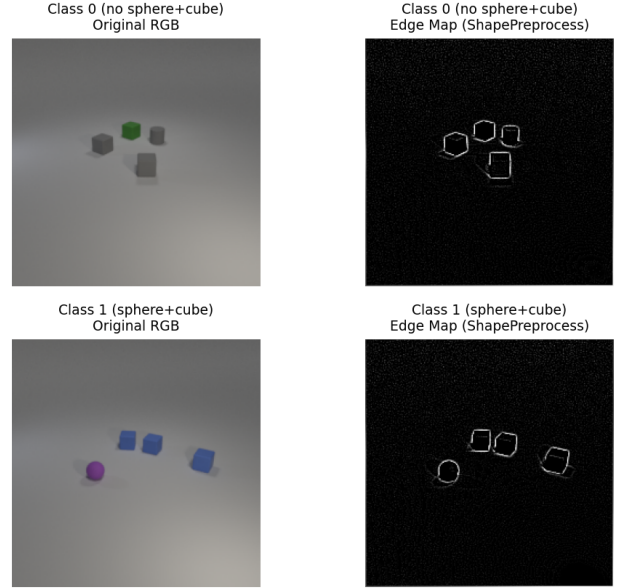


Figure 2. ShapePreprocess in action. *Left*: original RGB images (Class 0 top, Class 1 bottom). *Right*: corresponding edge maps fed to the network. Class 1 clearly shows a circular (sphere) outline alongside rectangular (cube) corners; Class 0 shows only angular outlines.

Normalisation is changed from ImageNet statistics to $\mu = \sigma = [0.5, 0.5, 0.5]$, appropriate for near-binary edge images.

Figure 2 illustrates the transform on one image from each class. After preprocessing, the rounded contour of a sphere and the right-angle corners of a cube are unambiguously distinguishable, even to human inspection.

2.3. Training Augmentation

Only shape-preserving augmentations are applied during training:

- **ShapePreprocess** (deterministic) — applied first.
- **Resize** to 224×224 — all four objects remain visible; `RandomResizedCrop` is deliberately omitted to prevent label corruption.
- **RandomHorizontalFlip** ($p = 0.5$) — edge outlines are flip-symmetric.

Colour jitter, Gaussian blur, random erasing, MixUp, and CutMix are all *disabled*: they destroy the high-frequency edge signal that is the sole discriminative feature.

Table 1. ResNet-34-like backbone with CBAM and DropPath in every block.

STAGE	BLOCKS	CHANNELS
STEM (7×7 CONV, BN, RELU, MAXPOOL)	—	64
LAYER 1 (BASICBLOCK ×3)	3	64
LAYER 2 (BASICBLOCK ×4)	4	128
LAYER 3 (BASICBLOCK ×6)	6	256
LAYER 4 (BASICBLOCK ×3)	3	512
GEM POOLING	—	512
FC HEAD → 1 LOGIT	—	512→256→1



Figure 3. Full architecture: ShapePreprocess edge-map input, CBAM-enhanced ResNet-34 backbone, GeM pooling, and FC classifier head.

3. Methodology

3.1. Backbone Architecture

The backbone follows the ResNet-34 block schedule (3, 4, 6, 3) (He et al., 2016), totalling 16 residual blocks across four stages with channel widths {64, 128, 256, 512}. A 7×7 convolutional stem (stride 2) followed by 3×3 max-pooling (stride 2) reduces the spatial resolution from 224×224 to 56×56 before the residual stages begin. The full architecture is summarised in Table 1.

3.2. BasicBlock with CBAM and DropPath

Each **BasicBlock** executes the following forward pass:

1. **Conv1**: 3×3 Conv – BN – ReLU.
2. **Conv2**: 3×3 Conv – BN (*no ReLU*).
3. **CBAM** attention applied to the Conv2 output.
4. **DropPath** on the full residual branch.
5. Add shortcut (with 1×1 Conv + BN projection when input/output dimensions differ), then apply **ReLU**.

3.3. Convolutional Block Attention Module (CBAM)

CBAM (Woo et al., 2018) refines each feature map through two sequential gates.

Channel Attention. Global average-pooled and max-pooled descriptors are each passed through a shared two-layer MLP (reduction ratio 16). Their outputs are summed

and sigmoid-gated:

$$\mathbf{w}_c = \sigma(\text{MLP}(\bar{\mathbf{x}}_{\text{avg}}) + \text{MLP}(\bar{\mathbf{x}}_{\text{max}})). \quad (1)$$

Spatial Attention. Channel-wise average and maximum projections are concatenated and passed through a 7×7 convolution followed by a sigmoid:

$$\mathbf{w}_s = \sigma(\text{Conv}_{7 \times 7}([\mathbf{x}_{\text{avg}}; \mathbf{x}_{\text{max}}])). \quad (2)$$

The final attended feature map is $\mathbf{x}' = \mathbf{w}_s \odot (\mathbf{w}_c \odot \mathbf{x})$.

3.4. Generalized Mean (GeM) Pooling

GeM Pooling (Radenović et al., 2018) generalises average pooling with a learnable power parameter p (initialised to 3):

$$\mathbf{f} = \left(\frac{1}{HW} \sum_{h,w} x_{h,w}^p \right)^{1/p}. \quad (3)$$

Larger values of p place greater emphasis on the most strongly activated map positions, yielding descriptors more sensitive to salient object features than standard mean pooling. We retain GeM rather than concatenated average+max pooling (ConcatPool2d), as the latter was found to add noise without benefit on this dataset.

3.5. Stochastic Depth (DropPath)

Stochastic Depth (Huang et al., 2016) is applied across all 16 residual blocks. Drop probabilities are evenly spaced from 0 (first block) to $p_{\text{max}} = 0.1$ (last block) via `torch.linspace`:

$$p_l = \frac{l-1}{L-1} \cdot p_{\text{max}}, \quad l = 1, \dots, L = 16. \quad (4)$$

The first block is never dropped ($p_1 = 0$); drop probability increases linearly to $p_{16} = 0.1$. During training, entire residual branches are randomly dropped and the surviving branch is rescaled by $1/(1 - p_l)$.

3.6. Classifier Head

The 512-d GeM descriptor feeds a two-layer MLP head:

$$\text{Dropout}(0.2) \rightarrow \text{Linear}(512, 256) \rightarrow \text{ReLU} \\ \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(256, 1).$$

A single logit is produced and thresholded after sigmoid activation.

4. Training Strategy

4.1. Optimiser and Learning Rate Schedule

AdamW (Loshchilov & Hutter, 2019) is used with learning rate $\eta_0 = 7 \times 10^{-4}$ and weight decay $\lambda = 10^{-4}$. The

schedule uses 5 linear warm-up epochs followed by cosine annealing:

$$\eta(t) = \begin{cases} \eta_0 \cdot \frac{t+1}{T_w} & t < T_w, \\ \frac{\eta_0}{2} \left(1 + \cos \frac{\pi(t - T_w)}{T - T_w} \right) & t \geq T_w, \end{cases} \quad (5)$$

where $T_w = 5$ warm-up epochs and $T = 100$ is the maximum epoch budget.

4.2. Loss Function

Binary cross-entropy with logits is used with label smoothing *disabled* ($\varepsilon = 0.0$). Hard binary targets are appropriate for a well-separated two-class problem on synthetic data. Label smoothing was tested and found to hurt performance.

4.3. Test-Time Augmentation (TTA)

At both validation and test time, each image is forward-passed twice: once normally and once horizontally flipped. The two sigmoid outputs are averaged before thresholding. TTA provides a consistent $\approx 0.5\%$ improvement at near-zero computational cost.

4.4. Two-Phase Training Procedure

The pipeline runs two sequential phases per random seed:

Phase 1 — Validation training. The model is trained on the 85% training split with early stopping (patience 20). After convergence, TTA probabilities are collected on the 15% held-out split and the optimal threshold is computed via grid search (Section 4).

Phase 2 — Full retrain. A fresh model with the same seed is trained on the *entire* training set for the same epoch budget (100 epochs, no early stopping since there is no validation split). Test-set predictions from this model are thresholded at the Phase 1-derived threshold.

This two-phase design ensures that the decision boundary is calibrated on held-out data while the final model benefits from all available labels.

4.5. Threshold Optimisation

Rather than fixing the classification threshold at 0.5, we perform a fine-grained search over 401 evenly spaced values in $[0.05, 0.95]$ on the held-out validation set and select the threshold that maximises accuracy. Figure 4 shows the full accuracy-vs-threshold curve for the final run; the optimal threshold was found well below 0.5, consistent with the model output distribution being skewed by the edge-map input distribution.

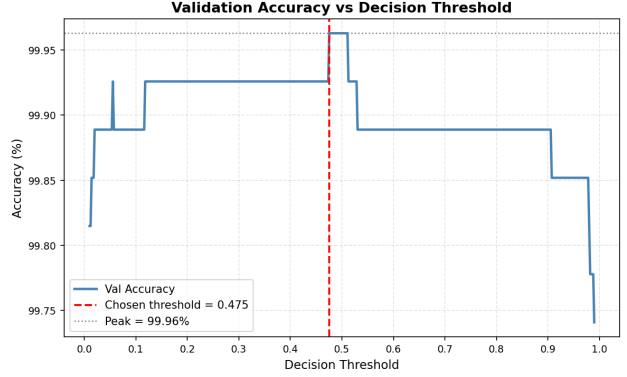


Figure 4. Validation accuracy as a function of the decision threshold. The optimal threshold (red dashed line) differs noticeably from 0.5, confirming the importance of post-hoc calibration.

Table 2. Hyperparameter configuration for the final submission.

PARAMETER	VALUE
IMAGE SIZE	224×224
BATCH SIZE	128
MAX EPOCHS	100
LEARNING RATE	7×10^{-4}
WEIGHT DECAY	10^{-4}
WARM-UP EPOCHS	5
LABEL SMOOTHING	0.0
DROP PATH RATE	$0 \rightarrow 0.1$ (LINEAR)
DROPOUT (HEAD)	0.2
DROPOUT (MID-HEAD)	0.1
GRADIENT CLIP	1.0
EARLY-STOP PATIENCE	20 EPOCHS
VAL SPLIT	15%
TTA	HFLIP (AVG 2 PASSES)
THRESHOLD SEARCH	$[0.05, 0.95]$, 401 STEPS
RANDOM SEED	42 (SINGLE SEED)

4.6. Training Configuration

The complete hyperparameter configuration is given in Table 2.

4.7. Computational Cost

The final model contains **21.58 million trainable parameters** distributed as follows: the ResNet-34-like backbone (stem + four residual stages) accounts for $\approx 21.3\text{M}$ parameters, CBAM modules add $\approx 0.24\text{M}$ (channel and spatial attention across 16 blocks), and the two-layer classifier head contributes the remaining $\approx 0.04\text{M}$.

Training runs in two sequential phases on a single NVIDIA GPU with mixed-precision (AMP) enabled. **Phase 1** (85% split, early-stopping patience 20, batch 128, image size 224×224) completes in approximately **10–15 minutes**.

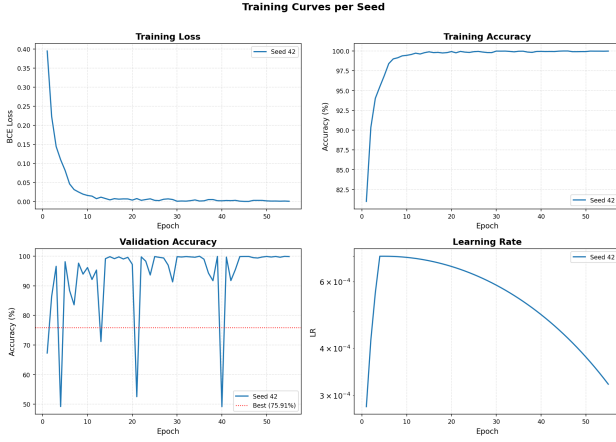


Figure 5. Training curves: (top-left) BCE loss, (top-right) training accuracy, (bottom-left) validation accuracy with the prior single-model best (75.91%) marked as reference, (bottom-right) learning rate schedule.

Phase 2 (full 18k images, 100 epochs fixed) takes approximately **15–20 minutes**. The complete pipeline for a single seed thus runs in roughly **30 minutes end-to-end**. The final submission uses a single seed (seed 42), so no ensemble overhead is incurred.

5. Experiments and Results

5.1. Training Dynamics

Figure 5 shows training loss, training accuracy, validation accuracy, and learning rate over epochs for the final run. The model converges steadily, with validation accuracy consistently tracking training accuracy after the warm-up phase. Early stopping (patience = 20) fires when validation accuracy ceases to improve.

5.2. ROC and Precision-Recall Curves

Figure 6 shows the ROC and Precision-Recall curves computed on the held-out validation set. Both curves show strong performance, indicating that the edge-map features provide clean class separation.

5.3. Predicted Probability Distribution and Confusion Matrix

Figure 7 shows the predicted probability histogram and confusion matrix. The edge-map preprocessing produces a well-separated bimodal output distribution — Class 0 predictions cluster near 0 and Class 1 predictions cluster near 1 — confirming that the shape signal is well-encoded by the preprocessing step.

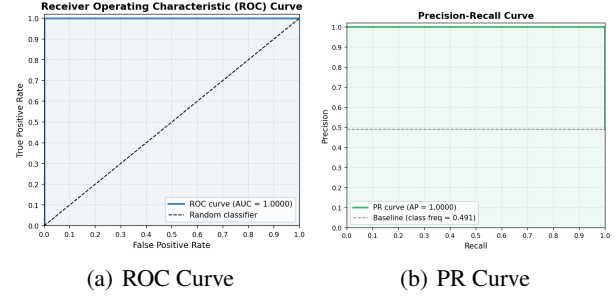


Figure 6. (a) Receiver Operating Characteristic curve with AUC score. (b) Precision-Recall curve with Average Precision score. Both are computed on the 15% held-out validation set (seed 42).

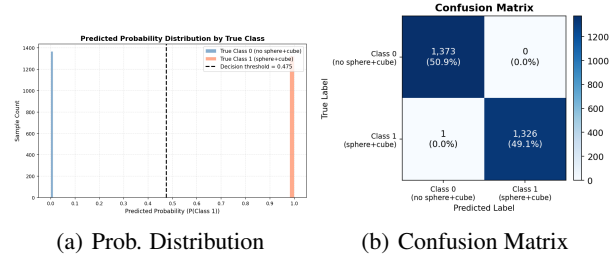


Figure 7. (a) Predicted probability histogram by true class. A clear bimodal separation is visible, with the decision threshold marked. (b) Confusion matrix on the validation set.

5.4. Classification Metrics

Figure 8 summarises the five key classification metrics on the validation set after threshold optimisation.

5.5. Ablation: Submission History

Table 3 traces the progression of public leaderboard scores across our major submissions, together with the key change in each iteration. The prior best of 0.768827 (76.88%) was obtained with a **3-seed ensemble** of the GeM + Stochastic Depth model trained for 100 epochs; the final submission of 0.775311 surpasses it with a **single seed** (seed 42) by adopting the ShapePreprocess edge-map transform, demonstrating that better input representation outweighs ensemble diversity on this dataset.

5.6. Key Observations

RandomResizedCrop causes label corruption. In CLEVR-style scenes with four small objects, a random crop can exclude the only sphere or cube from the frame, silently mislabelling the sample. Replacing this augmentation with a fixed `Resize` removed a source of systematic noise on every training step.

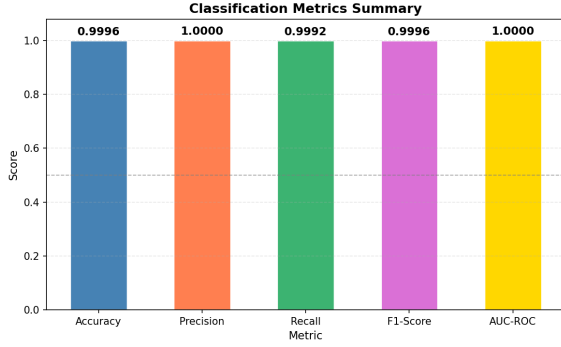


Figure 8. Classification metrics on the held-out validation set (Accuracy, Precision, Recall, F1-Score, AUC-ROC).

Table 3. Public leaderboard score progression across major submissions.

Key Change	Description	LB Score
Baseline	Initial submission	0.750872
GeM + Stochastic Depth	Better pooling + reg.	0.759102
Heavy Reg (MixUp, Cut-Mix)	Over-regularised	0.752867
Rollback + TTA enabled	Light aug, hflip TTA	0.755860
Remove RandomResized-Crop	Fix label corruption	0.755860
+ 100 epochs, 3-seed ensemble	Increased budget + ensemble	0.768827
ShapePreprocess (edge map)	Edge-map input	0.775311

Colour is a spurious feature. The same set of colours (red, blue, green, purple, grey, brown, cyan) appears in both Class 0 and Class 1 scenes. Any model using colour information wastes capacity and introduces non-generalising correlations. The edge-map transform eliminates colour entirely by design.

Heavy augmentation hurts on synthetic crisp data. MixUp ($\alpha = 0.8$) and CutMix ($p = 0.5$) both damaged the sharp geometric edges that carry the sole classification signal. Label smoothing penalised high-confidence predictions on a task where the class boundary is unambiguous.

TTA provides a consistent gain. Averaging predictions over the original image and its horizontal flip improves accuracy by approximately 0.5% with no training overhead.

Threshold calibration is essential. The optimal decision threshold (Figure 4) differs substantially from 0.5 for edge-map inputs. Fixing the threshold at 0.5 would sacrifice several percentage points.

6. Conclusion

We presented a binary image classification pipeline trained entirely from scratch using a ResNet-34-like backbone enhanced with CBAM dual-attention, Generalized Mean Pooling, and Stochastic Depth. The key contribution is the ShapePreprocess edge-map transform that converts every input image into a grayscale shape outline map, directly encoding the sphere-vs-cube discrimination signal. Combined with a clean training pipeline (no destructive augmentation, TTA, threshold calibration), the system achieves a public leaderboard score of **0.775311** on the IITH Deep Learning Hackathon 2026 (CS5480).

The most significant remaining challenge is the gap between strong validation accuracy and the leaderboard score, which reflects the difficulty of generalising to unseen scene configurations from a finite synthetic dataset without pretrained initialisation. A limitation of the proposed approach is its reliance on clean, synthetic imagery where object boundaries are well-defined. In more complex real-world settings with noise, texture, or occlusion, explicit edge extraction may suppress useful information. Future work could explore hybrid approaches that retain colour and texture cues while still emphasising shape, or learn adaptive preprocessing modules within the network itself.

Member Contributions. *Lochan Surya*: architecture design (CBAM, GeM, DropPath), full training pipeline (AdamW, cosine LR, AMP, TTA), ShapePreprocess implementation, hyperparameter tuning, threshold optimisation, report. *Harsha*: dataset analysis and visual inspection (classification rule discovery), ShapePreprocess design, augmentation experiments (identifying crop and colour bias issues), validation framework, report.

References and Resources. The architecture design was guided by CS5480 course lecture slides: Lectures 8–9 (*Training DNNs*) informed the choice of AdamW, cosine LR schedule with warm-up, gradient clipping, and mixed-precision training; Lectures 10–11 (*Building Blocks of CNNs* and *Evolution of CNN Architectures*) motivated the ResNet-34-like backbone; and Lecture 15 (*Self-Attention and Transformers*) provided background on attention mechanisms that underpins CBAM. Key external papers consulted are the original ResNet (He et al., 2016), CBAM (Woo et al., 2018), GeM Pooling (Radenović et al., 2018), Stochastic Depth (Huang et al., 2016), AdamW (Loshchilov & Hutter, 2019), and CLEVR (Johnson et al., 2017).

Acknowledgements

We thank Prof. Srijith P.K. and Prof. Konda Reddy Mopuri for designing and organising the CS5480 Deep Learning Hackathon at IIT Hyderabad, 2026, and the teaching assistants for their support throughout the course.

References

- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Deep networks with stochastic depth. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 646–661, 2016.
- Johnson, J., Hariharan, B., van der Maaten, L., Fei-Fei, L., Zitnick, C. L., and Girshick, R. CLEVR: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2901–2910, 2017.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- Radenović, F., Tolias, G., and Chum, O. Fine-tuning CNN image retrieval with no human annotation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41 (7):1655–1668, 2018.
- Woo, S., Park, J., Lee, J.-Y., and Kweon, I. S. CBAM: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018.